

README — corpus loading and sampling

This builds a corpus (from Wikipedia or your own files) and draws a **traceable sample** of a word, sentence, or document from it, recording every decision so the whole process can be audited.

The idea is simple: **load a corpus** → **choose what to observe** → **sample it carefully**, keeping the context around each find.

This folder contains:

File	What it is for
<code>corpus_loading.py</code>	Gets the texts and puts them all in one common shape.
<code>sampling.py</code>	Finds and samples words / sentences / documents.
<code>tracelog.py</code>	Keeps the record of every decision (the audit trail).
<code>exe.py</code>	The interactive program that asks you questions and runs everything.

To run: put the files in one folder and run `python exe.py` (from a terminal, not the VS Code Run button — the program asks you questions and needs a real terminal to type answers). You need Python 3, plus `requests` and `datasets`: `pip install requests datasets`.

The samples it produces are saved under `samples/<term>_<unit>/`.

1. `corpus_loading.py` — getting the texts

Motivation. Before analyzing anything you need a pile of texts, all stored the same way no matter where they came from. This file is the single front door.

The common shape. Every document is a name, its text (`body`), and notes about it (`meta`):
`{ "Title": { "body": "...", "meta": {"source": "wikipedia"} } }`

Functions

- `make_document(body, meta)` — Builds one document in the common shape.
- `validate_corpus(corpus)` — Checks a corpus is well-formed.
- `save_corpus(corpus, path)` / `load_corpus(path)` — Save/read a corpus, with a count summary at the top of the file.

Path 1 — Wikipedia web API (a few specific articles):

- `fetch_wikipedia_article(title)` — Downloads one article; waits and retries if Wikipedia says "too many requests".

- `load_from_wikipedia(titles)` — Downloads a list of titles.
- `search_wikipedia_titles(term)` / `search_and_load_wikipedia(term)` — Find and download articles mentioning a term.

Path 2 — your own files:

- `load_from_files(folder)` — Reads every `.txt` in a folder into documents.

Path 3 — large-scale Wikipedia sampling (recommended):

- `sample_wikipedia_dataset(term)` — Streams a full copy of Wikipedia and randomly samples articles that contain your term. Filters out junk: disambiguation pages ("X may refer to..."), lists, and trailing sections like "See also" / "References", keeping only the real article body. Helpers: `_clean_body` and `_is_disambiguation`.
-

2. `sampling.py` — finding and sampling

Motivation. Decide *what* counts as your observation (a word, a sentence, a document) and *how* to pick a sample of them. These two jobs are kept separate, so the same picking methods work for any kind of observation.

Choosing what to observe (extractors)

Each find remembers which document it came from and its exact position (its **offset**), which matters later for matching it to a model.

- `build_pattern(word, mode)` — Search rule for a word. Modes: `exact` (whole word only), `variants` (also short endings), `loose` (anywhere). Always ignores upper/lower case.
- `extract_segments` / `extract_sentences` / `extract_documents` — Find every word occurrence / every sentence / each whole document.
- `select_documents_with_word(corpus, word)` — Documents containing the word (first step of two-stage sampling).

Choosing how to pick (strategies)

These take any list of finds and return a sample:

- `exhaustive` — keep everything.
- `random_simple(n)` — `n` at random.
- `stratified(n, key)` — `n` keeping group proportions (e.g. by source).
- `reservoir(n)` — `n` at random from a huge/streaming list.

Controls (keeping the sample balanced)

- `cap_per_group(max, key)` — Limit how many finds each group (e.g. each article) contributes, so one article doesn't dominate.
- `cap_per_group_spaced(max, key, min_distance)` — Same, but the finds kept from one article are spread out (at least `min_distance` characters apart), so they come from different parts of the text. If the article is too short to space them, it **relaxes the distance gradually** so you still get the number you asked for. (This is what gives you diverse contexts instead of several occurrences from the same sentence.)
- `summary_per_group(key)` — Count how many finds each group gave.

Context window (KWIC)

- `extract_window(occurrence, mode, size)` — Adds the surrounding text: `left` and `right` of the word. Measured in `chars`, `words`, the whole `sentence`, or the whole `paragraph`. This turns a bare list of words into a readable concordance.

Saving and orchestrating

- `save_units` / `load_units` — Save/read finds, with a count summary. Used by every case below.
- `sample(...)` — A convenience for a single pass: one extraction + one strategy, saved and logged. This fits the **one-stage** cases (a sentence / document sample, or all occurrences of a word at once): extract the units, apply a strategy, add the window if needed, save.

The next two functions are **only for two-stage segment sampling** (the option where you first choose articles and then occurrences):

- `stage1_documents(corpus, word)` — Stage 1: pick the articles with the word, save that sub-corpus.
- `stage2_occurrences(subcorpus, word)` — Stage 2: find the word's occurrences inside those articles, cap/space them, add the context window.
- `full_flow(corpus, word)` — Runs stage 1 and stage 2 in a row.
- `print_kwic` — Prints a concordance.

So the structure depends on what you chose:

Case	How it is built
Segment, two stages	<code>stage1_documents</code> → <code>stage2_occurrences</code> (or <code>full_flow</code>)
Segment, one stage	<code>extract_segments</code> → strategy → <code>extract_window</code> , saved
Sentence	<code>extract_sentences</code> → strategy → saved
Document	<code>extract_documents</code> → strategy → saved

In the one-stage and sentence/document cases there is no separate "stage 1 sub-corpus": the units are taken from the corpus in a single pass. Only the two-stage segment option produces the intermediate `stage1_subcorpus.json`.

3. `tracelog.py` — the audit trail

Records every decision with its reason, so the work can be checked and reproduced.

- `record(operation, ..., justification, ...)` — Adds one entry: what was done, with which settings, **why**, a summary, the file it was saved to, and the time.
- `entries()` / `save(path)` — Read / save the whole record.

4. The exe (`exe.py`) — the interactive program

Motivation. Asks simple questions, runs the right parts based on your answers, and saves everything into its own folder so runs never overwrite each other.

Each run saves to `samples/<term>_<unit>/`. Sampling "china" goes to `samples/china_segment/`; "north korea" goes to `samples/north_korea_segment/`.

Output files in that folder

Which files appear depends on the case:

File	Contents	When
<code>corpus_full.json</code>	Every article loaded.	always
<code>stage1_subcorpus.json</code>	Articles that passed stage 1 (contain your word).	two-stage segment only
<code>stage2_kwic.json</code>	The final sample: each find with its context.	two-stage segment
<code>occurrences_kwic.json</code>	The final sample (single pass).	one-stage segment
<code><unit>_sample.json</code>	The final sample.	sentence / document
<code>tracelog.json</code>	The record of every decision.	always

(`stage1_subcorpus.json`) is an intermediate file for auditing — the *articles* the occurrences came from — not the final sample. It only exists when you pick the two-stage segment option.)

Every question, in order

1. **Which unit?** — `segment` (a word), `sentence`, or `document`.
2. **The target:**
 - segment: **word to search**, then **match type** (exact / variants / loose);
 - sentence/document: **expression it must contain** (empty = all).
3. **Where does the corpus come from?** — `wikipedia` or `manual`.
 - If Wikipedia: **how to fetch** — `dataset` (recommended) or `api`.
 - **Language** — `en` or `es`.
 - **If dataset** (these bring articles INTO THE CORPUS — the raw material, not the sampling yet):
 - **How many articles to fetch into the corpus** — default 300. More is better for diverse contexts; lower it for rare words, raise it for common ones.
 - **Minimum times the word must appear in an article** — default 2.
 - **How many Wikipedia articles to scan while fetching** — higher is more thorough but slower.
 - **Random seed**.
 - **If api:** how many articles to fetch.
 - **If manual:** paste each text and its source, blank line to finish.
4. **Restrict to one source?** — only asked if the corpus mixes sources.
5. **Of the articles that contain your word, how many to keep:**
 - **Minimum articles to keep** (fewest acceptable).
 - **Maximum articles to keep** (0 = keep all that have the word).

- **Minimum occurrences to keep** (fewest acceptable).
 - **Maximum occurrences to keep** (0 = no cap).
 - **Maximum occurrences per article** (0 = no cap) — caps how many finds each article contributes.
 - **Random seed.**
6. **Sampling approach** (segment only): **two stages** (choose articles, then occurrences) or **one stage** (all occurrences at once).
 7. **Strategy** — `exhaustive`, `random`, `reservoir`, and `stratified` if there are several sources. In two stages you pick one for articles and one for occurrences. If you pick a strategy that draws a fixed number (random / reservoir / stratified) but left the matching maximum as 0, it asks for that number now — and explains why.
 8. **Context window** (segment only): chars / words / sentence / paragraph; plus **window size** for chars/words. The **minimum distance between occurrences** is then set automatically from the window (so contexts don't overlap) and only reported, not asked.

Helpful warnings it gives

- **Per-article cap vs. target:** if your "maximum occurrences per article" makes the requested number of occurrences impossible (because $articles \times cap$ is smaller than what you asked), it warns you clearly, shows the arithmetic, and tells you how many articles — or what cap — you would need.
- **Not enough finds:** if a minimum can't be reached, it says so and continues with what it has, instead of failing.

At the end it prints the **TraceLog**: every step, with the reason, a summary, and the file each result was saved to.

Quick mental model

```

LOAD a corpus                (corpus_loading.py)   raw material, one shape
  ↓
CHOOSE what to observe      (sampling.py extractors)  words / sentences / docs
+ offsets
  ↓
SAMPLE it                   (sampling.py strategies + controls)  balanced,
traceable
  ↓
KEEP the context            (sampling.py window)    each find with text around it
  ↓
RECORD everything           (tracelog.py)           saved per run by exe.py

```

The samples land in `samples/<term>_<unit>/`.